

Appendix A — SQL Queries

Walmart Sales & Inventory Analysis

The following SQL queries correspond to the final analyses presented in the Power BI dashboard. Experimental and rejected approaches have been excluded.

Database used:

```
USE walmart_inventory;  
GO
```

A.1 Yearly Demand Trend

```
SELECT  
    YEAR(sale_date) AS sales_year,  
    SUM(CAST(units_sold AS BIGINT)) AS total_units_sold  
FROM dbo.sales_fact  
GROUP BY YEAR(sale_date)  
ORDER BY sales_year;
```

A.2 Store Performance by Demand

```
SELECT  
    store_id,  
    SUM(CAST(units_sold AS BIGINT)) AS total_units_sold  
FROM dbo.sales_fact  
GROUP BY store_id  
ORDER BY total_units_sold DESC;
```

A.3 Overall Category Demand Share

```
SELECT  
    cat_id,  
    SUM(CAST(units_sold AS BIGINT)) AS total_units_sold  
FROM dbo.sales_fact  
GROUP BY cat_id  
ORDER BY total_units_sold DESC;
```

A.4 Demand Volume vs Variability by Store

```
SELECT  
    store_id,  
    ROUND(AVG(CAST(units_sold AS FLOAT)), 2) AS avg_daily_demand,  
    ROUND(STDEV(CAST(units_sold AS FLOAT)), 2) AS demand_variability  
FROM dbo.sales_fact  
GROUP BY store_id  
ORDER BY avg_daily_demand DESC;
```

A.5 Holiday vs Regular Demand

```
SELECT  
    CASE  
        WHEN c.event_name_1 IS NOT NULL  
            OR c.event_name_2 IS NOT NULL  
        THEN 'Holiday/Event'  
        ELSE 'Regular Day'  
    END AS day_type,  
    SUM(CAST(s.units_sold AS BIGINT)) AS total_units_sold,
```

```

        AVG(CAST(s.units_sold AS FLOAT)) AS avg_daily_units
FROM dbo.sales_fact s
JOIN dbo.calendar c
    ON s.sale_date = c.date
GROUP BY
    CASE
        WHEN c.event_name_1 IS NOT NULL
            OR c.event_name_2 IS NOT NULL
        THEN 'Holiday/Event'
        ELSE 'Regular Day'
    END;

```

A.6 Store Year-over-Year Growth

```

WITH yearly_store_sales AS
(
    SELECT
        store_id,
        YEAR(sale_date) AS sales_year,
        SUM(CAST(units_sold AS FLOAT)) AS annual_units_sold
    FROM dbo.sales_fact
    GROUP BY
        store_id,
        YEAR(sale_date)
),
store_growth AS
(
    SELECT
        store_id,
        sales_year,
        annual_units_sold,
        LAG(annual_units_sold)
            OVER (
                PARTITION BY store_id
                ORDER BY sales_year
            ) AS previous_year_units
    FROM yearly_store_sales
)
SELECT
    store_id,
    sales_year,
    ROUND(annual_units_sold, 0) AS annual_units_sold,
    ROUND(previous_year_units, 0) AS previous_year_units,
    ROUND(
        (annual_units_sold - previous_year_units)
        * 100.0
        / NULLIF(previous_year_units, 0),
        2
    ) AS yoy_growth_pct
FROM store_growth
WHERE previous_year_units IS NOT NULL
ORDER BY
    store_id,
    sales_year;

```

A.7 Product Demand Concentration

```

WITH product_demand AS
(
    SELECT

```

```

        item_id,
        SUM(CAST(units_sold AS BIGINT)) AS total_units_sold
    FROM dbo.sales_fact
    GROUP BY item_id
),
ranked_products AS
(
    SELECT
        item_id,
        total_units_sold,
        ROW_NUMBER() OVER (
            ORDER BY total_units_sold DESC
        ) AS product_rank,
        SUM(total_units_sold) OVER () AS overall_demand
    FROM product_demand
)
SELECT
    item_id,
    product_rank,
    total_units_sold,
    ROUND(
        total_units_sold * 100.0 / overall_demand,
        2
    ) AS demand_share_pct,
    ROUND(
        SUM(total_units_sold) OVER (
            ORDER BY product_rank
            ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
        ) * 100.0 / overall_demand,
        2
    ) AS cumulative_demand_share_pct
FROM ranked_products
ORDER BY product_rank;

```

A.8 High-Demand Products with Highest Demand Variability

```

WITH store_product_metrics AS
(
    SELECT
        item_id,
        store_id,
        AVG(CAST(units_sold AS FLOAT)) AS avg_daily_units,
        STDEV(CAST(units_sold AS FLOAT)) AS demand_std_dev
    FROM dbo.sales_fact
    GROUP BY
        item_id,
        store_id
),
overall_average AS
(
    SELECT
        AVG(avg_daily_units) AS avg_demand
    FROM store_product_metrics
),
high_demand AS
(
    SELECT
        s.*
    FROM store_product_metrics s
    CROSS JOIN overall_average o
)

```

```

        WHERE s.avg_daily_units > o.avg_demand
    ),
    volatility_average AS
    (
        SELECT
            AVG(demand_std_dev) AS avg_std_dev
        FROM high_demand
    ),
    high_demand_high_volatility AS
    (
        SELECT
            h.*
        FROM high_demand h
        CROSS JOIN volatility_average v
        WHERE h.demand_std_dev > v.avg_std_dev
    )
SELECT TOP 15
    item_id,
    store_id,
    ROUND(avg_daily_units, 2) AS avg_daily_units,
    ROUND(demand_std_dev, 2) AS demand_std_dev
FROM high_demand_high_volatility
ORDER BY
    demand_std_dev DESC,
    avg_daily_units DESC;

```

A.9 Demand Intermittency

```

SELECT TOP 20
    item_id,
    cat_id,
    COUNT(DISTINCT sale_date) AS selling_days,
    SUM(units_sold) AS total_units_sold,
    ROUND(
        SUM(units_sold) * 1.0
        / COUNT(DISTINCT sale_date),
        2
    ) AS avg_units_per_selling_day
FROM dbo.sales_fact
WHERE units_sold > 0
GROUP BY
    item_id,
    cat_id
ORDER BY
    selling_days ASC,
    total_units_sold DESC;

```

A.10 Category Demand Trend Over Time

```

SELECT
    YEAR(s.sale_date) AS sales_year,
    s.cat_id,
    SUM(CAST(s.units_sold AS BIGINT)) AS total_units_sold
FROM dbo.sales_fact s
GROUP BY
    YEAR(s.sale_date),
    s.cat_id
ORDER BY
    sales_year,
    s.cat_id;

```

Power BI Measure Used for A.9

The intermittency visual expresses selling days as a percentage of the 1,913-day observation period.

```
Selling Day % =  
DIVIDE(  
    Demand_Intermittency[selling_days],  
    1913  
)
```